

How to remove only dangling (untagged and unused) images:

Run: in PowerShell or cmd

- `docker system df ---` It shows **how much disk space Docker is using** on your machine.
- `docker images -a ---` To list all images in your machine
- `docker image prune --` To remove only dangling images

What it does the docker image prune syntax

- Removes **dangling images** (unused, untagged images)
- Safe cleanup (won't remove images used by containers)

`docker image prune -a` → delete all unused images

Since you're low on storage, usually better to run:

- `docker system prune -a --` delete images, containers, network and cache

Keep in mind that when working with Docker, there is a high chance that the storage usage on the C: drive will increase significantly, as Docker images can exceed 30 GB when rebuilding images frequently.

- “`docker build -t imagename:tag .`”
- Docker creates **new layers**
- The old image becomes untagged (<none>)
- That old image → becomes a **dangling image**
- This happens especially if you rebuild frequently (CI/CD, local testing).

1) What are dangling images:

When you run:

- `docker images -a`

Dangling images look like:

<untagged> 33.8GB	1fa86cffba80
<untagged> 33.7GB	3c906f06f6a7
<untagged> 33.8GB	1fa86cffba80
<untagged> 33.7GB	3c906f06f6a7

These are **images without a tag** (no name, no version).

2. Intermediate build layers

During docker build, Docker creates temporary layers.

- If build changes or fails
- Some intermediate layers remain

These show up as dangling images

3. Updating tags

Example:

```
docker tag image1 myapp:v1
```

```
docker tag image1 myapp:latest
```

Later if latest is updated → old reference may lose tag → becomes dangling

4. Interrupted builds / failed builds

If:

- Build fails midway
- Or you cancel it

Partial layers/images remain → dangling

5. CI/CD pipelines (very relevant for you)

Since you work with builds:

- Each pipeline run builds new images
- Old ones lose tags
- Accumulate quickly

This is **very common in DevOps environments**

Where are these stored?

On Windows (Docker Desktop):

- Typically, under:

C:\ProgramData\Docker

You can check attached below images as references:

```
PS C:\> docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:58fc9524f99d6fc71e21cac572dec36a92f49e98c76dcd014d067666a0e09079
deleted: sha256:9e193f8cc2cbfd9478f8f3513a2aca1cade8c3a1239a5e0c06dbfe9dcdc68ec2
deleted: sha256:f525f3b28b3bd5546789854a31fb2c934ab6997779fba0479bb15a86faa0c75e
deleted: sha256:eb07db12293b337b72efdb27edb2d4c6eedfad4887c5fc5ea56b5072b80e48f6
deleted: sha256:93b5a97bcae5ed6c7443ca5d63e38b9869b84803aafbd5d9fc3a781bde18d363
deleted: sha256:5752ab17e069a01c103a5bf6aab76216491bacf7fa322c9e69f69f9cb55cd6af
deleted: sha256:37a78ad466c88d94727203927c69b42793c6f56ad45a6113bf271ae6cbeb466
deleted: sha256:c5130d38059c77eb524ea639b55e1096fbd9736cddb7e51b8f49794de1dd26ec
deleted: sha256:211810dd914c0cd861ae4a9923948933369570b20272c5bcc7fb1be8c15ef1b4
deleted: sha256:f83e3b8762faa493027dabfa2959a00219d40fcbeb9eb17b312e8bb56244c329
deleted: sha256:f3236a3c8955c3c53b45fee5ca1a5430fb86d4f4c278728f0c73fee6a6b434b5
deleted: sha256:9e8d864e6a3af3ea9a806e0fc95ba090b51b244b650ff12703848ba1c9c2727
```

Deleting all dangling images will free up disk space.